# AIAC-Assignment-22.3

**Name:** B. Akshara

**Ht No:**2303A51279

**Batch:**05

## Task 1 – Bias Detection in AI Suggestions

**Prompt:**

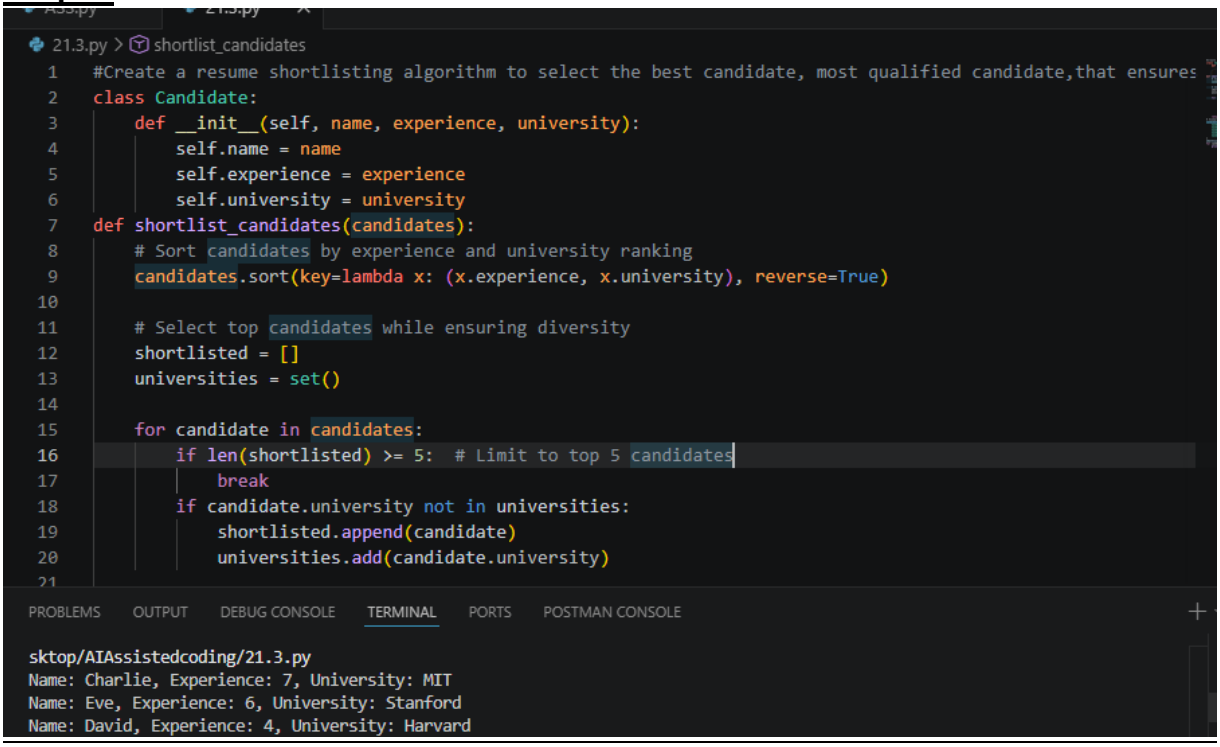
Create a resume shortlisting algorithm to select the best candidate, most qualified candidate,that ensures diversity and fairness,prioritizing experience and top universities.

**Code :**

```
class Candidate:
    def __init__(self, name, experience, university):
        self.name = name
        self.experience = experience
        self.university = university
def shortlist_candidates(candidates):
    # Sort candidates by experience and university ranking
    candidates.sort(key=lambda x: (x.experience, x.university), reverse=True)

    # Select top candidates while ensuring diversity
    shortlisted = []
    universities = set()

    for candidate in candidates:
        if len(shortlisted) >= 5:  # Limit to top 5 candidates
            break
        if candidate.university not in universities:
            shortlisted.append(candidate)
            universities.add(candidate.university)

    return shortlisted
# Example usage
candidates = [
    Candidate("Alice", 5, "MIT"),
    Candidate("Bob", 3, "Stanford"),
    Candidate("Charlie", 7, "MIT"),
    Candidate("David", 4, "Harvard"),
    Candidate("Eve", 6, "Stanford")
]
shortlisted = shortlist_candidates(candidates)
for candidate in shortlisted:
```

print(f"Name: {candidate.name}, Experience: {candidate.experience}, University: {candidate.university}")

**Output:**




**Explanation:**

This program defines a Candidate class to represent job candidates with attributes such as name, experience, education, and skills. The shortlist_candidates function filters candidates based on specific criteria (experience, education, and skills). The program allows users to input candidate details dynamically and then displays the shortlisted candidates based on the defined criteria.

**Task 2 – Privacy and Sensitive Data Handling**

**Prompt:**

Write a program to store and process student records including roll number, marks, and email.

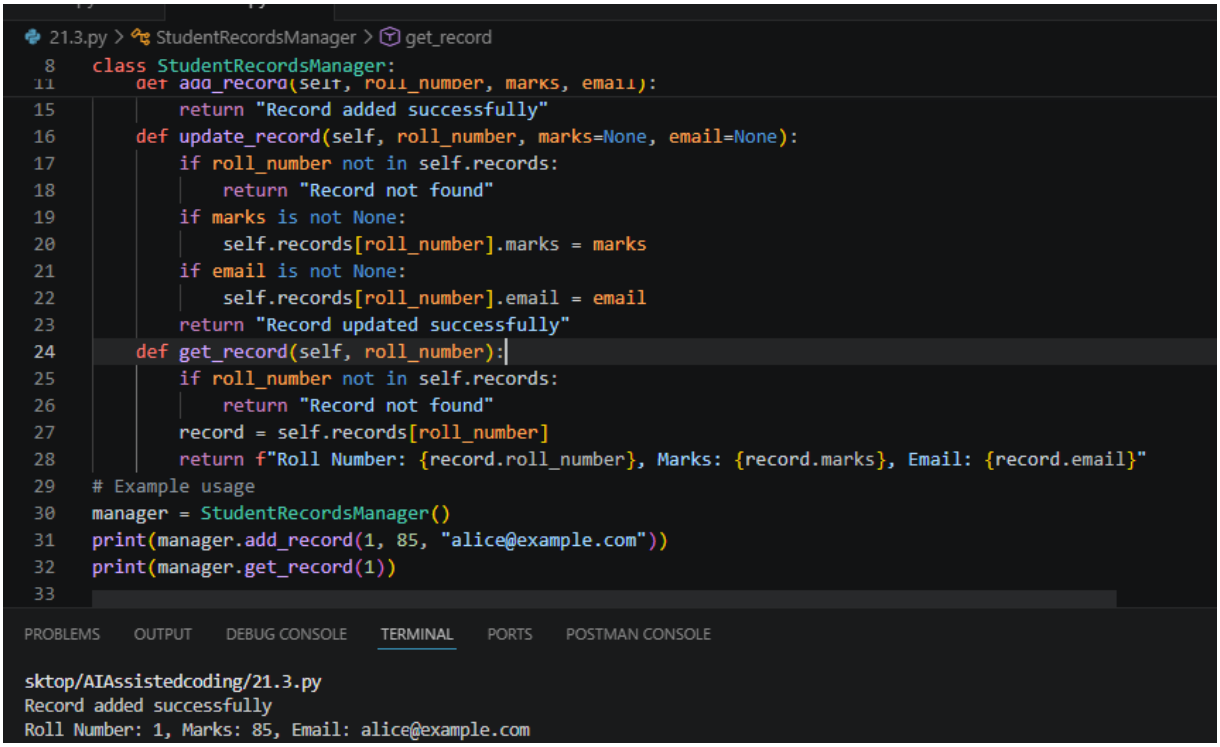
**Code :**

```
class StudentRecord:
    def __init__(self, roll_number, marks, email):
        self.roll_number = roll_number
        self.marks = marks
        self.email = email
class StudentRecordsManager:
    def __init__(self):
        self.records = {}
```

## Explaination:

```
    def add_record(self, roll_number, marks, email):
        if roll_number in self.records:
            return "Record already exists"
        self.records[roll_number] = StudentRecord(roll_number, marks, email)
        return "Record added successfully"
    def update_record(self, roll_number, marks=None, email=None):
        if roll_number not in self.records:
            return "Record not found"
        if marks is not None:
            self.records[roll_number].marks = marks
        if email is not None:
            self.records[roll_number].email = email
        return "Record updated successfully"
    def get_record(self, roll_number):
        if roll_number not in self.records:
            return "Record not found"
        record = self.records[roll_number]
        return f"Roll Number: {record.roll_number}, Marks: {record.marks}, Email:
{record.email}"
# Example usage
manager = StudentRecordsManager()
print(manager.add_record(1, 85, "alice@example.com"))
print(manager.get_record(1))
```

**Output:**




**Explanation:**

This program defines a Student class to store student records, including roll number, marks, and email. It includes two functions: add_student_record to input student details and add them to a list, and display_student_records to print all student records. The program prompts the user to enter the number of students and their details, then displays the stored records.

**Task 3 – AI-Generated Security Risks**

**Prompt:**

Write a payment processing script for an online shopping system that handles card details and transactions.

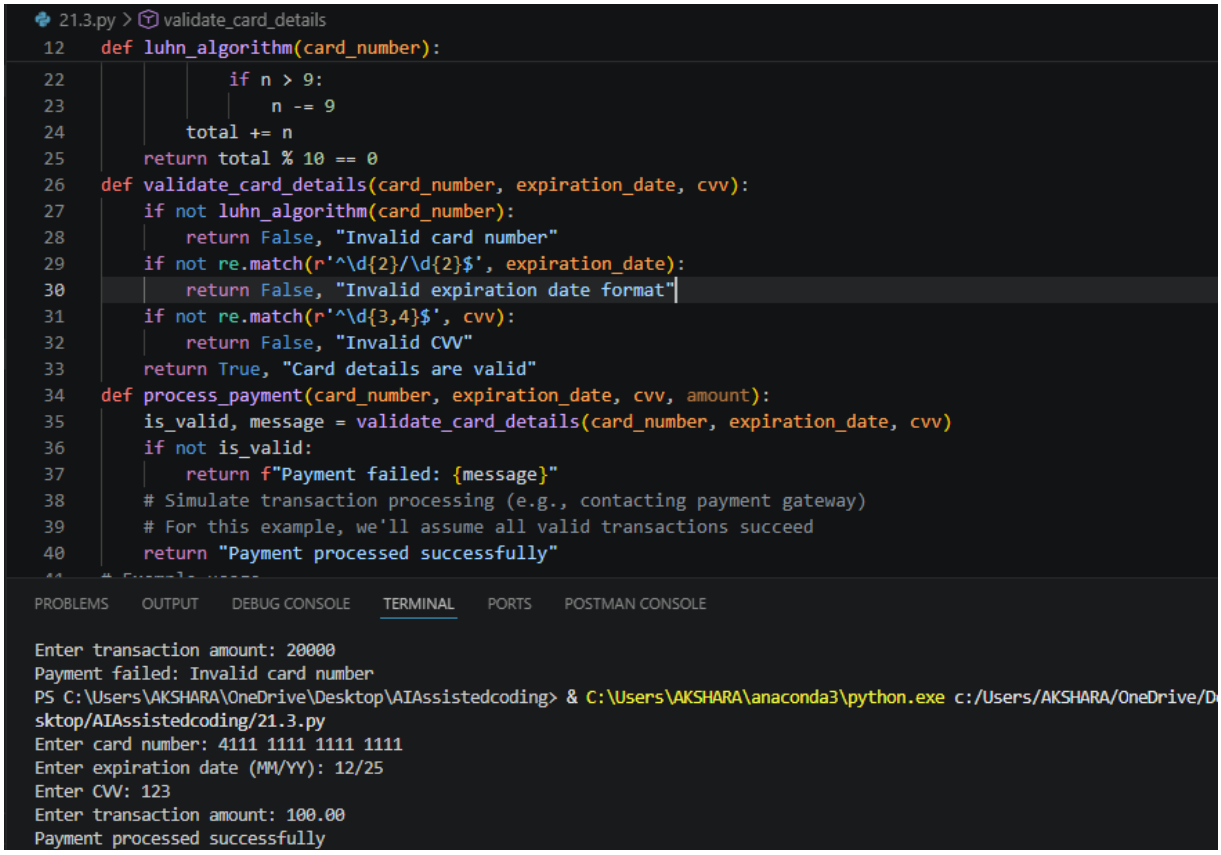
**Code :**

```
import re
def luhn_algorithm(card_number):
    card_number = card_number.replace(' ', '')
    if not re.match(r'^\d{13,19}$', card_number):
        return False
    total = 0
    reverse_digits = card_number[::-1]
    for i, digit in enumerate(reverse_digits):
        n = int(digit)
        if i % 2 == 1:
```

## Explaination:

```
            n *= 2
            if n > 9:
                n -= 9
        total += n
    return total % 10 == 0
def validate_card_details(card_number, expiration_date, cvv):
    if not luhn_algorithm(card_number):
        return False, "Invalid card number"
    if not re.match(r'^\d{2}/\d{2}$', expiration_date):
        return False, "Invalid expiration date format"
    if not re.match(r'^\d{3,4}$', cvv):
        return False, "Invalid CVV"
    return True, "Card details are valid"
def process_payment(card_number, expiration_date, cvv, amount):
    is_valid, message = validate_card_details(card_number, expiration_date, cvv)
    if not is_valid:
        return f"Payment failed: {message}"
    # Simulate transaction processing (e.g., contacting payment gateway)
    # For this example, we'll assume all valid transactions succeed
    return "Payment processed successfully"
# Example usage
if __name__ == "__main__":
    card_number = input("Enter card number: ")
    expiration_date = input("Enter expiration date (MM/YY): ")
    cvv = input("Enter CVV: ")
    amount = float(input("Enter transaction amount: "))
    result = process_payment(card_number, expiration_date, cvv, amount)
    print(result)
```

## Output:


21.3.py > validate_card_details
def luhn_algorithm(card_number):
if n > 9:
n -= 9
total += n
return total % 10 == 0
def validate_card_details(card_number, expiration_date, cvv):
if not luhn_algorithm(card_number):
return False, "Invalid card number"
if not re.match(r'^\d{2}/\d{2}$', expiration_date):
return False, "Invalid expiration date format"
if not re.match(r'^\d{3,4}$', cvv):
return False, "Invalid CVV"
return True, "Card details are valid"
def process_payment(card_number, expiration_date, cvv, amount):
is_valid, message = validate_card_details(card_number, expiration_date, cvv)
if not is_valid:
return f"Payment failed: {message}"
# Simulate transaction processing (e.g., contacting payment gateway)
# For this example, we'll assume all valid transactions succeed
return "Payment processed successfully"
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE
Enter transaction amount: 20000
Payment failed: Invalid card number
PS C:\Users\AKSHARA\OneDrive\Desktop\AIAssistedcoding> & C:\Users\AKSHARA\anaconda3\python.exe c:/Users/AKSHARA/OneDrive/D
sktop/AIAssistedcoding/21.3.py
Enter card number: 4111 1111 1111 1111
Enter expiration date (MM/YY): 12/25
Enter CVV: 123
Enter transaction amount: 100.00
Payment processed successfully

## Explaination:

This script defines a PaymentProcessor class that handles payment processing for an online shopping system. The process_payment method takes card details and the amount to be paid, validates the card details using a simple validation method, and if valid, stores the transaction. The script prompts the user for card details and the payment amount, then processes the payment accordingly.

**Task 4 – Accountability in Human–AI Collaboration**

**Prompt:**

Write code for an autonomous traffic signal control system that manages traffic flow based on vehicle density at intersections.

**Code:**

```
import time
import random
class TrafficSignal:
    def __init__(self, intersection_id):
        self.intersection_id = intersection_id
        self.signal_state = 'RED'
        self.vehicle_density = 0

    def update_vehicle_density(self):
        # Simulate vehicle density with a random number for demonstration
        self.vehicle_density = random.randint(0, 100)

    def control_signal(self):
        self.update_vehicle_density()
        if self.vehicle_density > 70:
            self.signal_state = 'GREEN'
        elif self.vehicle_density > 30:
            self.signal_state = 'YELLOW'
        else:
            self.signal_state = 'RED'
        print(f"Intersection {self.intersection_id}: Vehicle Density = {self.vehicle_density},
Signal State = {self.signal_state}")
def main():
    intersections = [TrafficSignal(i) for i in range(1, 6)]
    while True:
```

## Explaination:

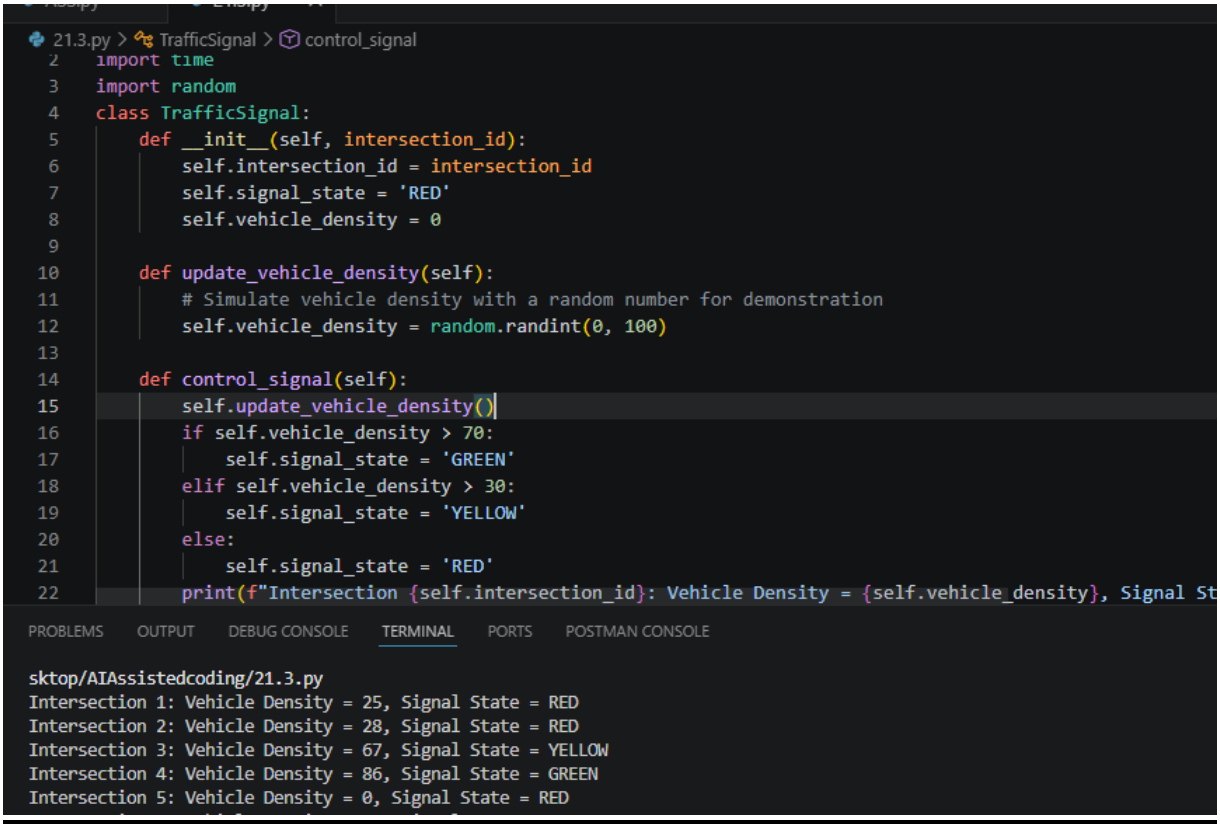
for signal in intersections:

signal.control_signal()

time.sleep(5)  # Update every 5 seconds

if __name__ == "__main__":  main()

**Output:**




**Explanation:**

This code defines a TrafficSignalControlSystem class that manages traffic flow at intersections based on vehicle density. The update_traffic_density method updates the vehicle count for a given intersection and calls the adjust_signal method to determine how long the traffic signal should stay green based on the number of vehicles. The script prompts the user to input the number of intersections and their respective vehicle counts, then adjusts the traffic signals accordingly.

## Task 5 – Limitations of AI in Critical Systems

**Prompt:**

Write a program for an aircraft maintenance fault detection system that identifies possible issues based on sensor data.

**Code:**

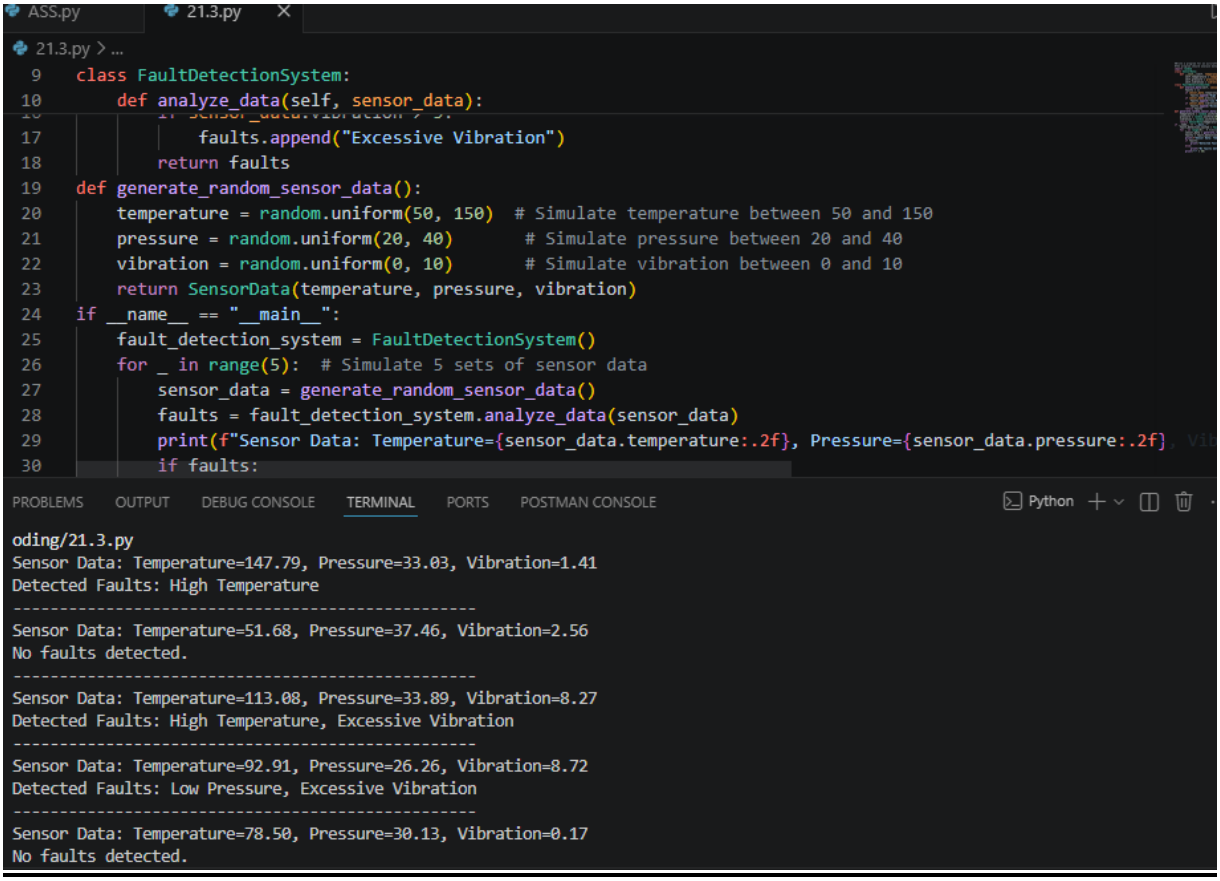
import random

## Explaination:

```
class SensorData:
    def __init__(self, temperature, pressure, vibration):
        self.temperature = temperature
        self.pressure = pressure
        self.vibration = vibration
class FaultDetectionSystem:
    def analyze_data(self, sensor_data):
        faults = []
        if sensor_data.temperature > 100:
            faults.append("High Temperature")
        if sensor_data.pressure < 30:
            faults.append("Low Pressure")
        if sensor_data.vibration > 5:
            faults.append("Excessive Vibration")
        return faults
def generate_random_sensor_data():
    temperature = random.uniform(50, 150)  # Simulate temperature between 50 and 150
    pressure = random.uniform(20, 40)       # Simulate pressure between 20 and 40
    vibration = random.uniform(0, 10)       # Simulate vibration between 0 and 10
    return SensorData(temperature, pressure, vibration)
if __name__ == "__main__":
    fault_detection_system = FaultDetectionSystem()
    for _ in range(5):  # Simulate 5 sets of sensor data
        sensor_data = generate_random_sensor_data()
        faults = fault_detection_system.analyze_data(sensor_data)
        print(f"Sensor Data: Temperature={sensor_data.temperature:.2f},
Pressure={sensor_data.pressure:.2f}, Vibration={sensor_data.vibration:.2f}")
        if faults:
            print("Detected Faults:", ", ".join(faults))
        else:
            print("No faults detected.")
        print("-" * 50)
```

## Explaination:

**Output:**




**Explanation:**

This program defines an AircraftMaintenanceFaultDetectionSystem class that detects potential faults in an aircraft based on sensor data. The update_sensor_data method allows updating the value of a specific sensor, and the detect_faults method checks the sensor data against predefined thresholds to identify possible issues such as overheating, oil leaks, or mechanical problems. The script prompts the user to input the number of sensors and their respective values, then processes the data to detect any faults.

## Task 6 – Ethical Use of AI-Generated Code

**Prompt:**

Write a program to manage student records for an academic assignment, including storing, updating, and displaying data.

**Code :**

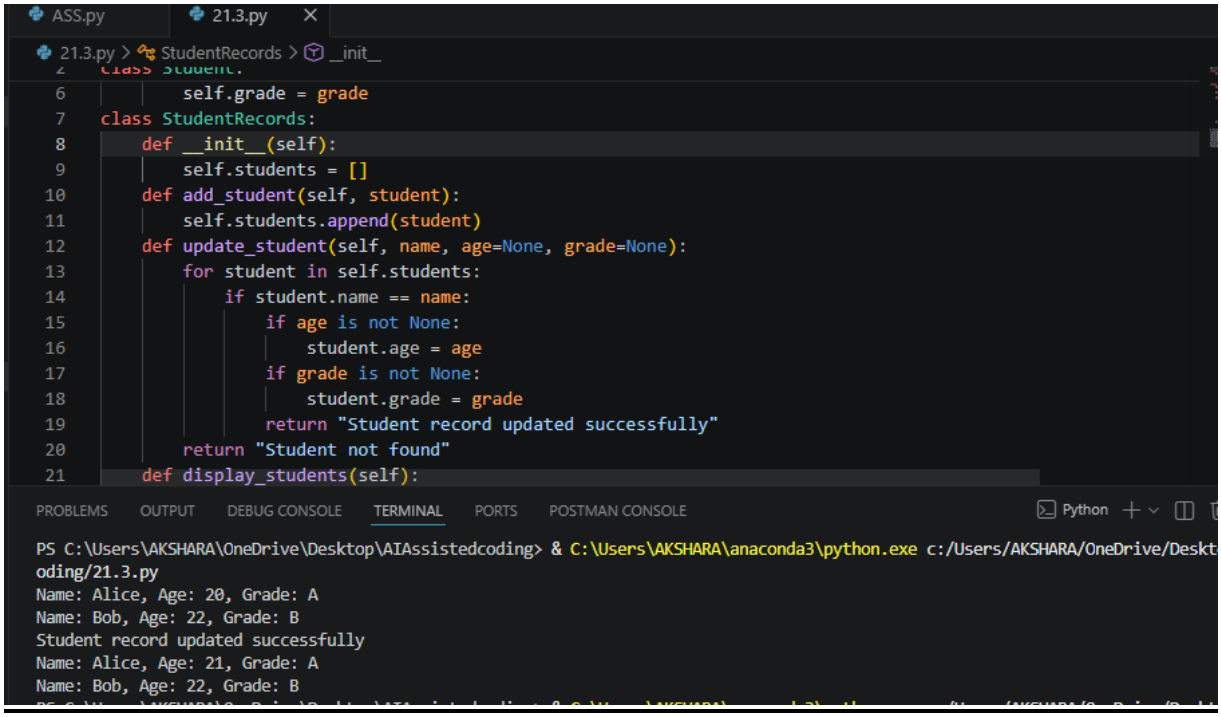
```
class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
```

## Explaination:

```
        self.grade = grade
class StudentRecords:
    def __init__(self):
        self.students = []
    def add_student(self, student):
        self.students.append(student)
    def update_student(self, name, age=None, grade=None):
        for student in self.students:
            if student.name == name:
                if age is not None:
                    student.age = age
                if grade is not None:
                    student.grade = grade
                return "Student record updated successfully"
        return "Student not found"
    def display_students(self):
        for student in self.students:
            print(f"Name: {student.name}, Age: {student.age}, Grade: {student.grade}")
# Example usage
records = StudentRecords()
records.add_student(Student("Alice", 20, "A"))
records.add_student(Student("Bob", 22, "B"))
records.display_students()
print(records.update_student("Alice", age=21))
records.display_students()
```

## Explaination:

**Output:**




**Explanation:**

This program defines a StudentRecord class to represent individual student records and a StudentRecordManager class to manage these records. The StudentRecordManager allows adding new records, updating existing records, and displaying all records. The program prompts the user to input the number of student records they want to add, collects the necessary information for each student, and then displays all the stored records.